Why Cypress?

Cypress is a testing tool that bills itself as “fast, easy, and reliable testing for anything that runs in a browser.” (In case you haven’t noticed yet, I’m always fascinated by how these tools describe themselves.) It is committed to making testing stable in JavaScript-land, and as a newer tool, it has taken design lessons from older tools. It can run from a command line and also has a test runner application that allows you to run tests interactively. Rails integration isn’t perfect, but it’s manageable.

Capybara is the traditional end-to-end testing tool in Rails.[79] And like Cypress, Capybara also allows you to interact with a simulated browser. (You can read all about testing JavaScript and Rails using Capybara in *Rails 5 Test Prescriptions*.)

Cypress has a couple of advantages over Capybara and one significant disadvantage. The disadvantage is that Cypress is not as good as Capybara in integrating with our Rails code. Capybara tests can use RSpec and be run as part of a normal Rails testing process. Although it’s discouraged, Capybara tests can access ActiveRecord for setting up data or making assertions about data after the fact.

Cypress tests are written in JavaScript and run in their own process. The cypress-rails gem allows us some access to Rails for setup, but overall, it’s still a more awkward setup in Cypress than in Capybara. That’s the downside.

On the upside, Cypress has a different architecture than Capybara. Cypress is effectively in control of the entire run process, including the browser process. Cypress has its own testing UI that we can use to choose which tests run or use to have tests auto-run on file changes. Because Cypress controls the run process, it’s more robust than Capybara when waiting for actions to complete, and it’s better able to provide debugging tools and automated screenshots and videos of tests. Cypress can even simulate the server and provide fake responses if you’d like.